Task_8_RSLOS.py

Для использования скрипта для РСЛОС необходимо в блоке:

if __name__ == '__main__':

отредактировать значение переменных **'proverb'**, **'key1', 'key2'и 'key3'**, а затем - просто запустить сам скрипт.

По итогу в консоли будут выведены:

1. Первый шифротекст и его расшифровка;
2. Второй шифротекст и его расшифровка.

```
PS C:\Users\Thunboo> python -u "c:\Users\Thunboo\Desktop\Крипта\Task_8_RSLOS.py"
ШФЦУО ЧЗЬЭР АБЖГР ЫВВОЩ КАЯЕН ЩНУСХ ЖЬКЛД
ЛЕОПА РДНЕМ ОЖЕТИ ЗМЕНИ ТЬСВО ИХПЯТ ЕНТЧК

ВЗЪФБ ЛЫКАЫ ДЯВГИ БКЧИБ ДЯЖЬА ГЫЬНЭ ЗНЮЬО
ЛЕОПА РДНЕМ ОЖЕТИ ЗМЕНИ ТЬСВО ИХПЯТ ЕНТЧК
PS C:\Users\Thunboo>
```

Код программы приложен в приложении А.

Приложение А

```python
#  Вариант 5  #
# Бройдо Глеб #
#    ИБ-31    #
#     2025    #
###############

##############################################################################
# text_utils ---------------------------------------------------------------#
ALPHABET = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"                                #
PROVERB  = "Леопард не может изменить своих пятен."                         #
                                                                            #
#--------------Функции для "стандартизации" строки под шифрование--------------#
def remove_special_chars(text: str) -> str:                                 #
    '''                                                                     #
    Убирает пробелы. Заменяет точки и запятые на ТЧК и ЗПТ.                 #
                                                                            #
    Также делает все буквы заглавными.                                      #
    '''                                                                     #
    new_text = []                                                           #
    text = text.upper()                                                     #
    for i in text:                                                          #
        new_text.append(                                                    #
            i.replace(',', 'ЗПТ').replace('.', 'ТЧК').replace(' ', '')      #
        )                                                                   #
    return ''.join(new_text)                                                #
                                                                            #
def insert_spaces(text: str) -> str:                                        #
    '''                                                                     #
    Форматирует строку в вид:                                               #
                                                                            #
    5 символов - пробел - 5 символов - пробел ...                           #
    '''                                                                     #
    new_text = []                                                           #
    for i, v in enumerate(text):                                            #
        new_text.append(v)                                                  #
        if (i + 1) % 5 == 0:                                                #
            new_text.append(' ')                                            #
    return ''.join(new_text)                                                #
                                                                            #
def standartise(text: str) -> str:                                          #
    '''                                                                     #
    Форматирует строку в строку без пробелов и с заменой на ТЧК и ЗПТ.      #
                                                                            #
    Затем - приводит к виду: 5 символов - пробел - 5 символов - пробел ...  #
    '''                                                                     #
    return insert_spaces(remove_special_chars(text))                        #
#---------------------------------------------------------------------------#
##############################################################################


# Number - текущие биты в РСЛОС
def rshift(number: str, key: str) -> str:
    new_bit = sum([1 if key[i] == number[i] == "1" else 0 for i in range(len(key))]) % 2
    return str(new_bit) + number[:-1]

def lshift(number: str, key: str) -> str:
    new_bit = sum([1 if key[i] == number[i] == "1" else 0 for i in range(len(key))]) % 2
    return number[1:] + str(new_bit)
```

```python
def generate_bits(length: int, start_value: str, key: str, shift: str="right") -> str:
    if shift == "right":
        out_bit = -1
        shift = rshift
    elif shift == "left":
        out_bit == 0
        shift = lshift

    temp_key = start_value[out_bit]
    value = start_value
    for _ in range(length):
        value = shift(value, key)
        temp_key += value[out_bit]
    return temp_key

def check_period(gamma: str) -> str:
    '''
    Считает период получившейся последовательности битов.
    '''
    for i in range(1, len(gamma)//2):
        flag = 1
        for j in range(len(gamma)//i - 1):
            if gamma[j*i: (j+1)*i] != gamma[(j+1)*i: (j+2)*i]:
                flag = 0
        if flag:
            return i
    return 0

def bits_XOR(all_bits: list) -> str:
    temp_bits = ""
    for i in range(len(all_bits[0])):
        temp_bits += str(sum([int(all_bits[j][i]) for j in range(len(all_bits))]) % 2)
    return temp_bits

def symbol_to_bits(symbol: str) -> str:
    '''
    Возвращает номер символа в алфавите в двоичной репрезентации.
    '''
    return f"{ALPHABET.find(symbol):05b}"

def string_to_bits(text: str) -> str:
    '''
    Переводит посимвольно целую строку в ее битовую репрезентацию.
    '''
    new_text = []
    for i in text:
        new_text.append(symbol_to_bits(i))
    return ''.join(new_text)

def bits_to_symbol(bits: str) -> str:
    '''
    Возвращает саму букву из ее бинарной кодировки.
    '''
    return ALPHABET[int(bits, base=2)]

def bits_to_string(bits: str, block_size: int=5) -> int:
    '''
    Переводит поблочно (5 битов) целую последовательность битов в строку символов.
    '''
    new_text = []
    for i in range(0, len(bits), block_size):
        new_text.append(bits_to_symbol(bits[i:i + block_size]))
    return ''.join(new_text)

def encrypt_decrypt_rslos(text: str, start_values: list, keys: list, shift:
str="right") -> str:

    # Исходный текст в битовой репрезентации
```

```python
    bits = string_to_bits(text)
    generated_bits = []
    for i in range(len(keys)):
        generated_bits.append(
            generate_bits(len(bits), start_values[i], keys[i], shift)
        )
    gamma = bits_XOR(generated_bits)
    bits = bits_XOR([bits, gamma])

    return bits_to_string(bits)

if __name__ == "__main__":
    proverb = PROVERB
    text_for_encryption = remove_special_chars(proverb)

    # Для первого задания ключ 25 имеет период 14
    key1 = f"{25:05b}"
    start_val1 = "10110"
    # Для второго задания комбинация ключей 19 и 1 имеет период 40 > 38
    key2 = f"{19:05b}"
    start_val2 = "11100"
    key3 = f"{1:05b}"
    start_val3 = "00011"

    # Шифротекст 1-го задания
    print(insert_spaces(encrypt_decrypt_rslos(text_for_encryption, [key1], [start_val1])))
    print(insert_spaces(
            encrypt_decrypt_rslos(
                encrypt_decrypt_rslos(
                    text_for_encryption
                  , [key1]
                  , [start_val1]
                  )
                  , [key1]
                  , [start_val1]
                )
            )
        )

    print()

    # Шифротекст 2-го задания
    print(insert_spaces(encrypt_decrypt_rslos(text_for_encryption, [key2, key3], [start_val2, start_val3])))
    print(insert_spaces(
            encrypt_decrypt_rslos(
                encrypt_decrypt_rslos(
                    text_for_encryption
                  , [key1, key2]
                  , [start_val1, start_val2]
                  )
                  , [key1, key2]
                  , [start_val1, start_val2]
                )
            )
        )
```